

Logic Programming

Rodica Potolea
Camelia Lemnaru
Richard Ardelean

Lecture #9

Side effects

Agenda

- **Built in predicates**
 - **Side Effects:** assert **and** retract
- Utilization on complex tasks
 - Towers of Hanoi

Side effects

- Allow for persistent structure
- Allow for wider scope visibility (modelling global variables; created in some predicate and visible elsewhere, without passing arguments)
- Allow for metaprogramming
- By using them, we exceed the pure Logic Programming (operations NOT undone on backtracking)

Side effects

- Built in predicates
- **assert/retract** for adding/extracting ONE clause of the predicate with the name provided as argument
- The argument (for both assert and retract) is a variable representing a dynamic predicate
 - `assert(X) / retract(X)`
- The variable *X* must be ALREADY instantiated for the predicate to be updated

Side effects – assert

- **assert** comes into 2 flavors:
 - **asserta** – adds in the beginning (on top) of the existing clauses for the predicate provided as argument
 - **assertz** / **assert** – adds at the end (at bottom) of the existing clauses for the predicate provided as argument
- It NEVER fails
- on backtracking, the item (clause) is NOT removed. However, **assert** does not re-execute on backtracking (NO new clause is added).

Side effects – retract

- removes the first item (clause of the predicate provided as argument) with whom it matches successfully
- if no match (the matching fails), retract fails. So, as opposed to assert, it can fail
- on backtracking, the item (clause removed) is NOT added back
 - On the contrary, another item is removed in case another successful match is possible, **or** retract fails

assert

- Assume we have some (3 in the ex.) clauses for a predicate $p(\dots)$ (arguments do not matter here).
- Here on the right, the clauses have numbers added to represent their index

$p(\dots) : -\dots$

$p(\dots) : -\dots$

$p(\dots) : -\dots$

$p_1(\dots) : -\dots$

$p_2(\dots) : -\dots$

$p_3(\dots) : -\dots$

asserta / assertz

- After the execution of `asserta (p (...))`, the predicate becomes:

$p_{\text{new}} (...) : - \dots$

$p_1 (...) : - \dots$

$p_2 (...) : - \dots$

$p_3 (...) : - \dots$

- After the execution of `assertz (p (...))`, the predicate becomes:

$p_1 (...) : - \dots$

$p_2 (...) : - \dots$

$p_3 (...) : - \dots$

$p_{\text{new}} (...) : - \dots$

Assert example

```
:-dynamic p/1.
```

No result is actually shown, it just results in a true.

```
p(1).
```

```
?- listing(p/1).
```

```
p(2).
```

```
p(1).
```

```
p(3).
```

```
p(2).
```

```
p(4).
```

```
p(3).
```

```
p(5).
```

```
p(4).
```

```
p(5).
```

```
q:- assertz(p(6)), fail.
```

```
?- q, listing(p/1).
```

```
q.
```

```
p(1).
```

```
p(2).
```

```
p(3).
```

```
p(4).
```

```
p(5).
```

```
p(6).
```

```
?- q.
```

error (No permission to modify static procedure 'p/1')

```
true.
```

retract

- Assume this scenario:

$p_1 (...) : -...$

$p_2 (...) : -...$

$p_3 (...) : -...$

- The execution of `retract(p(...))` would:
 - Start scanning the list of clauses of the predicate p
 - Stop at the first clause where arguments (not mentioned here) match
 - Remove the given clause and report success
 - If no successful matching, the retract fails

retract – success on the first clause

- Assume the same scenario:

$p_1 (...) : -...$

$p_2 (...) : -...$

$p_3 (...) : -...$

- If execution of `retract (p (...))` succeeds on matching with **first** clause of p , it leads to:

$p_2 (...) : -...$

$p_3 (...) : -...$

retract – success on the second clause

- Assume the same scenario:

$p_1 (...) : -...$

$p_2 (...) : -...$

$p_3 (...) : -...$

- If execution of `retract (p (...))` succeeds on matching with **second** clause of p , it leads to:

$p_1 (...) : -...$

$p_3 (...) : -...$

retract – success on the third clause

- Assume the same scenario:

$p_1 (...) : -...$

$p_2 (...) : -...$

$p_3 (...) : -...$

- If execution of `retract (p (...))` succeeds on matching with **third** clause of p , it leads to:

$p_1 (...) : -...$

$p_2 (...) : -...$

retract – fails

- Assume the same scenario:

$p_1 (...) : -...$

$p_2 (...) : -...$

$p_3 (...) : -...$

- If execution of `retract(p(...))` does NOT succeed trying to match p to ANY of the 3 clauses, the result would be:

$p_1 (...) : -...$

$p_2 (...) : -...$

$p_3 (...) : -...$

retract – on backtracking

- After the execution of retract, on backtracking, NO undo is performed (the removal stays). That clause is NEVER restored, unless an explicit request (assert) is made
- Moreover, on the attempt to REsatisfy the goal (due to backtracking), Prolog continues FROM THE PLACE it previously removed and tries to remove ANOTHER clause, and:
 - it does so if the match succeeds,
 - or else fails and the execution is resumed (backtracked) to the predicate on the left of retract

Computer Science

retract – on backtracking - contd

- Assume the same scenario where we have 5 clauses:

$p_1 (...) : -...$

$p_2 (...) : -...$

$p_3 (...) : -...$

$p_4 (...) : -...$

$p_5 (...) : -...$

- Assume the execution of `retract(p(...))` succeeds on matching with the **second** clause of predicate $p_2(...)$, after execution the predicate contains:

$p_1 (...) : -...$

$p_3 (...) : -...$

$p_4 (...) : -...$

$p_5 (...) : -...$

retract – on backtracking - contd

- If now we backtrack again, and retract matches the 3rd clause (i.e. $p_3 (...)$ which is the second actually), the predicate looks:

$p_1 (...)$: -...

$p_4 (...)$: -...

$p_5 (...)$: -...

- If we backtrack again, and retract matches the 5th clause (i.e. $p_5 (...)$ which is the third actually) the predicate looks:

$p_1 (...)$: -...

$p_4 (...)$: -...

- A new backtrack now would fail, as we reached the end of the list of clauses.

Retract example

```
:-dynamic p/1.
```

No result is actually shown, it just results in a true.

```
p(1) .
```

```
p(2) .
```

```
p(3) .
```

```
p(4) .
```

```
p(5) .
```

```
?- listing(p/1) .
```

```
p(1).
```

```
p(2).
```

```
p(3).
```

```
p(4).
```

```
p(5).
```

```
q:- retract(p(_)), fail.
```

```
q.
```

```
?- q, listing(p/1) .
```

```
?- q.
```

```
true.
```

error (No permission to modify static procedure 'p/1')

```
true.
```

retract – on backtracking - contd

- Assume the same scenario where we have 5 clauses:

$p_1 (...) : -...$

$p_2 (...) : -...$

$p_3 (...) : -...$

$p_4 (...) : -...$

$p_5 (...) : -...$

- Assume the execution of `retract(p(_))` cannot unify with ANY of the 5 clauses, the execution fails and `p` remains unchanged:

$p_1 (...) : -...$

$p_2 (...) : -...$

$p_3 (...) : -...$

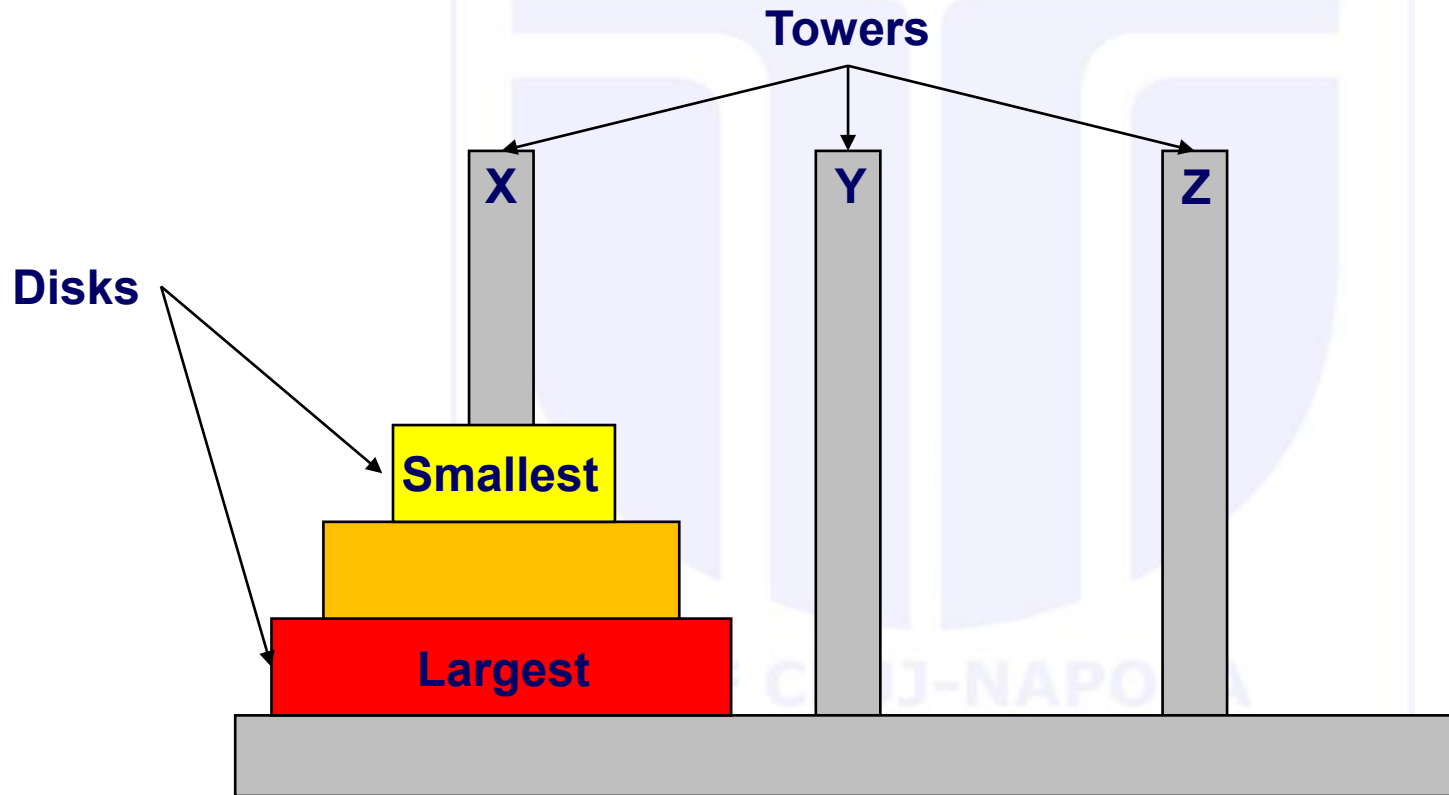
$p_4 (...) : -...$

$p_5 (...) : -...$

assert and retract

- Combining assert and retract
 - `asserta + retract => LIFO`
 - `assertz + retract => FIFO`

Towers of Hanoi – the problem

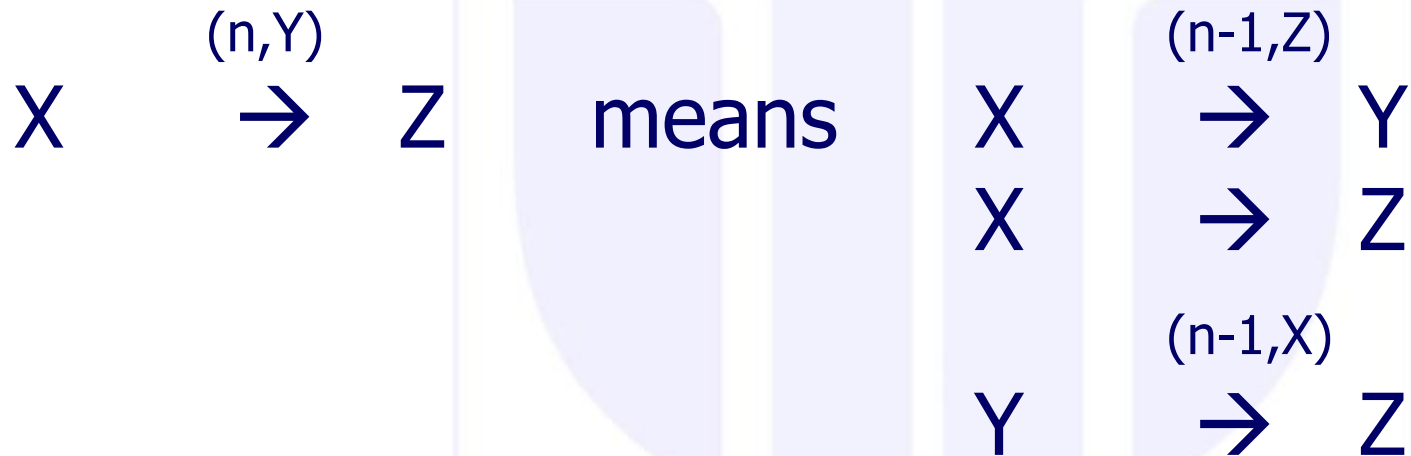


Towers of Hanoi - formulation

- Given n disks and 3 towers
- Move the disks from tower X to Z (same order – smallest to largest)
- At any time, the constraints are:
 - One disk at a time
 - Just the top disk is accessible
 - On any tower, any disk has the diameter smaller than the diameter of the disk on top of which it stays

Towers of Hanoi - rephrasing

Start with n disks on X and end with n disks on Z



- Means:

- Move n discs from X to Z using Y as intermediate by means of:
 - Move $(n-1)$ discs from X to Y using Z as intermediate
 - Move one single disc from X to Z
 - Move $(n-1)$ discs from Y to Z using X as intermediate

Towers of Hanoi - rephrasing

Start with n disks on X and end with n disks on Z

$$\begin{array}{ccc} X & \xrightarrow{(n,Y)} & Z \end{array} \quad \text{means} \quad \begin{array}{ccc} X & \xrightarrow{(n-1,Z)} & Y \\ X & \xrightarrow{\quad} & Z \\ Y & \xrightarrow{(n-1,X)} & Z \end{array}$$

BUT this is almost Prolog code:

```
move_discs(N,X,Y,Z):-
    N1 is N-1,
    move_discs(N1,X,Z,Y),
    move_disc(X,Z),
    move_discs(N1,Y,X,Z).
```

Towers of Hanoi - realization

- Disc representation (as stack of facts in the knowledge base):

`x(1) .`

`⋮`

`x(n-1) .`

`x(n) .` % n is NOT a variable, it is a value.

- How is it realized? % `init_tower/2 (+Input, +List)`

`init_tower(_,0):-!.`

`init_tower(T,N):-`

`D=..[T,N],` % T name of tower, as x in example; N size

`asserta(D),` % put on top

`N1 is N-1,`

`init_tower(T,N1) .`

Towers of Hanoi – moving one disk

- Move one disc from Input Pole to Output Pole:

```
move_disc(IP,OP):-
```

```
    DI=..[IP,Size],      % IP name of input pole, Size diam of disk
    retract(DI),!,       % extract from top
    DO=..[OP,Size],      % IP name of output pole, Size diam of disk
    asserta(DO).         % put on top
```

- If input has:

```
x(3).
x(4).      y(6).
x(5).      y(7).
```

- And call `move_disc(x,y)` . goes to:

```
    y(3).
x(4).      y(6).
x(5).      y(7).
```

Towers of Hanoi – complete code

```

move_discs(0,_,_,_):-!.
move_discs(N,X,Y,Z):-
    N1 is N-1,
    move_discs(N1,X,Z,Y),
    move_disc(X, Z),
    move_discs(N1,Y,X,Z).

move_disc(IP,OP):-
    DI=..[IP,Size],
    retract(DI),!,
    DO=..[OP,Size],
    asserta(DO).

init_tower(_,0):-!.
init_tower(T,N):-
    D=..[T,N],
    asserta(D),
    N1 is N-1,
    init_tower(T,N1).

:- dynamic(input/1).
:- dynamic(int/1).
:- dynamic(out/1).

% INITIAL call
hanoi(N):-
    init_tower(in,N),
    move_discs(N,in,int,out).

[q] ?- retractall(out(_)),
hanoi(3), listing(in/1),
listing(out/1).

```